

Avant l'outil : une autorisation vérifiable

Autorisation écrite → action minimale → preuve expurgée → vérification → reset.

Autorisé : app.shoplabs.test + API · Interdit : hôte, conteneurs d'autrui, services externes.

01

Borner cible, durée et actions

02

Distinguer fait, hypothèse et conclusion

03

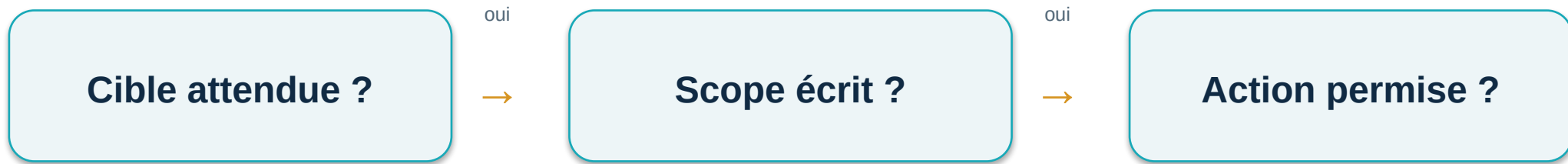
Revenir à un état propre

ShopLab : dedans, dehors, arrêt immédiat

CIBLE OBSERVÉE	DÉCISION	POURQUOI
<code>app.shoplab.test</code>	CONTINUER	cible nommée, compte synthétique
<code>api.shoplab.test</code>	CONTINUER	API explicitement incluse
<code>host.docker.internal</code>	ARRÊTER	machine hôte hors périmètre
<code>example.org</code>	ESCALADER	service externe inattendu

Règle : une cible inattendue n'est jamais testée pour « voir ce que c'est ».

Décider : continuer, arrêter ou escalader



Après les trois questions, appliquer une seule décision :

3 × oui

au moins 1 × non

information absente

CONTINUER

ARRÊTER

ESCALADER

STOP = aucune requête supplémentaire · conserver une trace minimale · prévenir le responsable

Une preuve est un dossier lisible par un tiers

DOSSIER

preuves/TP01/

├ 00_scope.md

├ 01_environnement.txt

├ 02_requete_http.txt

└ index_sha256.txt

UNE LIGNE D'INDEX EXPLIQUE LA PREUVE

UTC

2026-09-21T08:42:00Z

ACTION

curl .../api/health

RÉSULTAT

HTTP 200 + X-Correlation-ID

LIMITE

Limite : ne prouve pas le checkout

Même résultat, qualité de preuve différente

PREUVE FAIBLE

- ✗ Capture : « 200 OK »
- ✗ commande absente
- ✗ cible et heure inconnues
- ✗ conclusion : « tout est sûr »

même 200



PREUVE UTILE

- ✓ commande + cible locale
- ✓ UTC + commit
- ✓ sortie minimale expurgée
- ✓ conclusion et limite

Démo : préparer la preuve avant l'action

TERMINAL · LAB LOCAL UNIQUEMENT

```
$ date -u +%FT%TZ  
$ ./scripts/verify-host.sh  
$ git rev-parse --short HEAD
```

EXTRAIT SYNTHÉTIQUE

```
[PASS] Docker + Compose v2  
[PASS] curl / OpenSSL / Python 3  
commit abc1234 (exemple) · sortie  
variable
```

Avant toute suite :
FAIL bloque · WARN se vérifie

Au tableau : que faire face à l'inattendu ?

Une redirection pointe vers example.org. Continuer, arrêter ou escalader ?

A

Continuer une fois

B

Arrêter + preuve

C

Scanner le service

Réponse : B · ne pas suivre · noter URL + UTC · prévenir le responsable

S01 → Diagnostic + TP01-A

Diagnostic D01–D24 → TP01 mise en place → TP01-A scope et dossier de preuve

1 35 min

Diagnostic D01–D24

2 20 min

Vérifier et démarrer

3 20 min

TP01-A : scope + preuves

PREUVES

00_scope.md · sortie verify-host expurgée · index + preuve de reset

STOP : cible externe · secret réel · FAIL hôte · reset incertain

Une requête traverse huit étapes observables

Browser → DNS → TCP → TLS → proxy → API → base → réponse

Même requête · même X-Correlation-ID · observations distinctes à chaque frontière.

01

Ordonner DNS, TCP, TLS et HTTP

02

Relier proxy, API et base

03

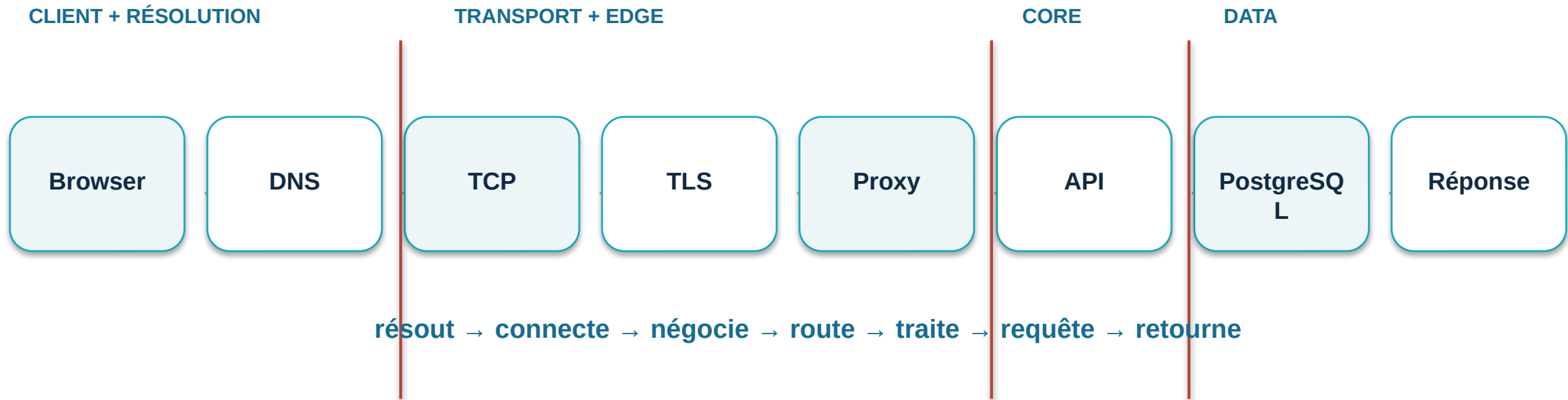
Localiser une panne par couche

Chaque couche répond à une question différente

COUCHE	OBSERVATION	CE QUE CELA PROUVE
DNS	127.0.0.1	le nom local est résolu
TCP	connect 8080	un listener accepte la connexion
HTTP	GET /api/health	le proxy reçoit une intention
API/DB	SELECT 1	health atteint la base, pas le checkout

Règle : une observation répond à une question de couche — jamais à toutes les questions.

ShopLab : trajet aller et retour



Aller : requête + ID → Retour : statut + headers + corps · les logs gardent UTC + ID

Frontières rouges : poste / edge / core / données

Un identifiant relie proxy, API et base

MÊME IDENTIFIANT

X-Correlation-ID

└ s02-groupe-07

└ proxy 08:54:01.120

└ api 08:54:01.126

└ db 08:54:01.129

CHRONOLOGIE SYNTHÉTIQUE

PROXY

08:54:01.120 proxy

API

08:54:01.126 API

DB

08:54:01.129 DB

RETOUR

08:54:01.132 HTTP 200

Dépanner une couche à la fois

PREUVE FAIBLE

- ✗ redémarrer tout
- ✗ changer DNS + TLS + proxy
- ✗ mélanger des request IDs
- ✗ conclure depuis un seul 200

1 variable



PREUVE UTILE

- ✓ garder le même chemin
- ✓ tester une frontière
- ✓ corréler ID + UTC
- ✓ noter ce qui reste non prouvé

Démo : corrélér la requête sans deviner

TERMINAL · LAB LOCAL UNIQUEMENT

```
$ curl -sv -H 'X-Correlation-ID: s02-demo' \
$ http://127.0.0.1:8080/api/health
```

EXTRAIT SYNTHÉTIQUE

```
> GET /api/health HTTP/1.1
< HTTP/1.1 200 OK
< X-Correlation-ID: s02-demo
{"status":"ok"} · synthétique
```

Limite :
health 200 ≠ checkout vérifié

Au tableau : où la chaîne casse-t-elle ?

TCP se connecte, mais le certificat ne correspond pas au nom. Quelle couche ?

A DNS

B TLS

C API

Réponse : B · vérifier SNI, SAN, chaîne et horloge · ne pas toucher à l'API

S02 → TP01-B/D + TD01

TP01-B : requête HTTP → TP01-D : proxy/API/base → TD01 : hypothèses falsifiables

1 35 min

TP01-B : HTTP

2 45 min

TP01-D : proxy → DB

3 46 min

TD01 : lancement

PREUVES

trace HTTP annotée · ID corrélé proxy/API · schéma + hypothèse testable

STOP : cible externe · secret réel · FAIL hôte · reset incertain

HTTP est un contrat; le navigateur ajoute ses politiques

Requête HTTP → décision serveur → réponse → décision du navigateur

Même réponse HTTP; lecture autorisée ou bloquée selon origine, credentials et headers.

01

**Interpréter méthodes, statuts
et headers**

02

Distinguer cookie et session

03

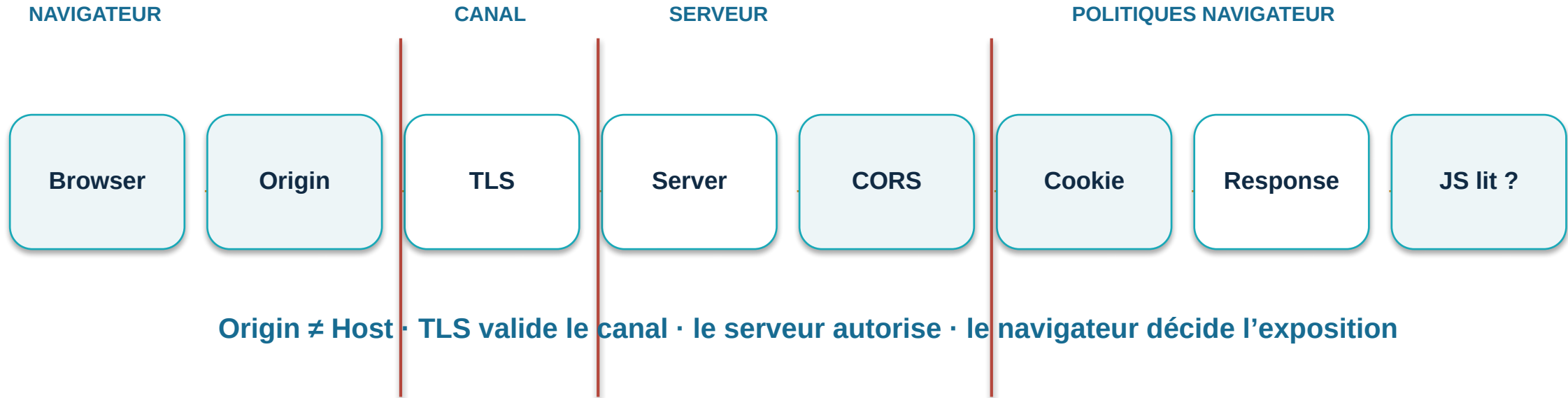
Séparer SOP, CORS et CSP

Lire les champs avant d'interpréter

CHAMP	EXEMPLE	RÔLE / LIMITE
Host	api.shoplabs.test	sélectionne l'hôte virtuel
Origin	https:// app.shoplabs.test	décrit l'origine du script
Cookie	session=[EXPUR GÉ]	est joint selon portée/attributs
Status	200 OK	décrit HTTP, pas l'autorisation métier

Règle : lire le champ, nommer son rôle, puis seulement interpréter la décision.

Host route; Origin décrit l'appelant



SOP/CORS ne remplace jamais l'autorisation serveur

Requête à gauche, décision à droite

REQUÊTE HTTP

REQUÊTE

- └ GET /api/profile
- └ Host: api.shoplab.test
- └ Origin: https://app...
- └ Cookie: [EXPURGÉ]

RÉPONSE + POLITIQUES

STATUS

200 OK

TYPE

Content-Type: application/json

COOKIE

Set-Cookie: __Host-session=...

POLICY

CSP + CORS + no-store

Headers : avant / après durcissement

AVANT

- ✗ cookie sans Secure
- ✗ Origin réfléchi
- ✗ CSP absente
- ✗ conclusion depuis curl seul

même flux



APRÈS + LIMITES

- ✓ Secure + HttpOnly + SameSite
- ✓ origines exactes
- ✓ CSP adaptée + no-store
- ✓ navigateur + curl comparés

Démo : serveur visible, décision navigateur distincte

TERMINAL · LAB LOCAL UNIQUEMENT

```
$ curl -i -H 'Origin: https://evil.test' \  
$ http://127.0.0.1:8080/api/health
```

EXTRAIT SYNTHÉTIQUE

```
HTTP/1.1 200 OK  
Content-Type: application/json  
Access-Control-Allow-Origin:  
(absent)  
curl reçoit; le navigateur décide ·  
synthétique
```

Limite :
curl reçoit ≠ JavaScript autorisé

Au tableau : que prouve réellement curl ?

curl reçoit HTTP 200 sans header CORS. Le JavaScript cross-origin peut-il lire la réponse ?

A

Oui, status 200

B

Non, CORS absent

C

Oui, curl réussit

Réponse : B · curl reçoit; le navigateur bloque la lecture au script

S03 → TP02-A

TP02-A : capturer login → annoter requête/réponse → évaluer cookie et politiques navigateur

1 35 min

TP02-A : capturer

2 45 min

Annoter + comparer

3 46 min

Vérifier + preuves

PREUVES

headers annotés · cookie expurgé évalué · limite navigateur/curl

STOP : cible externe · secret réel · FAIL hôte · reset incertain

Menaces et risque

Passer d'un dessin d'architecture à des scénarios testables et priorisés.

01

Identifier actifs et entrées

02

**Dessiner les frontières
de confiance**

03

**Relier menace, contrôle
et test**

Palette pédagogique provisoire · charte officielle À confirmer

Trois repères avant l'outil

01

**Une frontière
marque un
changement de
confiance**

Décision observable

02

**OWASP guide le
raisonnement, pas
le verdict**

Limite à expliciter

03

**Un risque dépend
du contexte**

Preuve à conserver

Le flux et ses décisions



À chaque flèche : qui fait confiance à qui ? quelle donnée ? quel contrôle ? quelle preuve ?

Cas guidé : faits avant verdict

Le frontend et l'API partagent le même laptop mais pas la même responsabilité.

FAIT OBSERVÉ

Deux processus et interfaces distincts créent une frontière même sur un hôte unique.

RISQUE À TESTER

Un catalogue de vulnérabilités sans actif ni preuve ne priorise rien.

Règle : une hypothèse utile indique l'observation qui pourrait la réfuter.

Vulnérable -> corrigé -> vérifié

ÉTAT VULNÉRABLE

- X Lister OWASP
- X Attribuer un score seul
- X Dessiner sans flux



ÉTAT CORRIGÉ

- OK Nommer l'actif
- OK Formuler l'abus
- OK Définir un test falsifiable

Même test négatif + parcours légitime positif + risque résiduel explicite

Démo : Construire un scénario acteur -> actif -> contrôle

TERMINAL LOCAL

```
$ printf '%s\n' 'acteur=client' 'action=lit objet tiers' 'actif=profil' 'contrôle=owner check' 'test=200/403'
```

SORTIE ATTENDUE

six champs reliés: acteur, action, actif, impact, contrôle et test

Prédire -> exécuter -> lire -> conclure -> retester -> reset

Checkpoint de compréhension

Quelle observation réfuterait votre scénario de BOLA ?

1 · Position

Formulez votre réponse

2 · Preuve

Quelle observation la soutient ?

3 · Limite

Qu'est-ce qui reste non prouvé ?

Passage à la pratique supervisée

TD02 + jalon capstone: DFD, quatre scénarios d'abus et tests associés.

PREUVES À REMETTRE

- 1** DFD légendé
- 2** frontières justifiées
- 3** test négatif/positif

STOP CONDITION

**Cible inattendue
Secret réel
Reset incertain**

Authentification et session

Concevoir un cycle d'identité qui résiste avant, pendant et après la connexion.

01

Expliquer hash, MFA et login

02

Suivre création, rotation et révocation

03

Comparer cookie de session et JWT

Palette pédagogique provisoire · charte officielle À confirmer

Trois repères avant l'outil

01

Le hash protège au repos

Décision observable

02

MFA réduit un risque, sans tout résoudre

Limite à expliciter

03

Logout doit invalider le contexte serveur

Preuve à conserver

Le flux et ses décisions



À chaque flèche : qui fait confiance à qui ? quelle donnée ? quel contrôle ? quelle preuve ?

Cas guidé : faits avant verdict

Un identifiant de session reste inchangé avant et après authentification.

FAIT OBSERVÉ

La continuité de l'identifiant rend plausible une fixation de session.

RISQUE À TESTER

Ajouter une expiration longue ne corrige pas l'absence de rotation.

Règle : une hypothèse utile indique l'observation qui pourrait la réfuter.

Vulnérable -> corrigé -> vérifié

ÉTAT VULNÉRABLE

- X Stockage réversible
- X Token éternel
- X Logout côté écran



ÉTAT CORRIGÉ

- OK Hash adapté
- OK Rotation à l'authn
- OK Révocation et délai borné

Même test négatif + parcours légitime positif + risque résiduel explicite

Démo : Observer les changements du contexte d'authentification

TERMINAL LOCAL

```
$ curl -i -c /tmp/session.cookies -b /tmp/session.cookies  
http://127.0.0.1:8080/api/health
```

SORTIE ATTENDUE

headers Set-Cookie/Cookie observables ou absence explicitement documentée

Prédire -> exécuter -> lire -> conclure -> retester -> reset

Checkpoint de compréhension

Que doit invalider un logout fiable ?

1 · Position

Formulez votre réponse

2 · Preuve

**Quelle observation la
soutient ?**

3 · Limite

**Qu'est-ce qui reste non
prouvé ?**

Passage à la pratique supervisée

TP02-B: observer login, expiration, rotation et logout; proposer deux tests négatifs.

PREUVES À REMETTRE

- 1 avant/après login
- 2 rejet après logout
- 3 aucun secret dans la preuve

STOP CONDITION

**Cible inattendue
Secret réel
Reset incertain**

Autorisation et BOLA

Décider côté serveur qui peut faire quoi sur quel objet, à chaque requête.

01

Distinguer authn et authz

02

**Comparer RBAC et
décision objet**

03

Prouver 200/401/403

Palette pédagogique provisoire · charte officielle À confirmer

Trois repères avant l'outil

01

**Le rôle ne prouve
pas la propriété**

Décision observable

02

**Le client contrôle
l'identifiant
demandé**

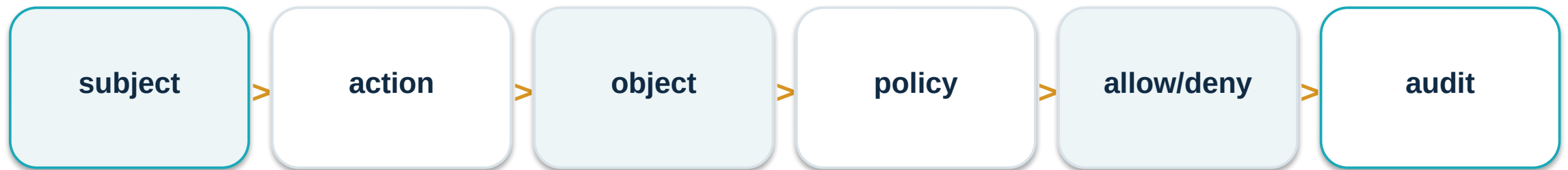
Limite à expliciter

03

**Chaque objet exige
une décision
serveur**

Preuve à conserver

Le flux et ses décisions



À chaque flèche : qui fait confiance à qui ? quelle donnée ? quel contrôle ? quelle preuve ?

Cas guidé : faits avant verdict

Alice change /users/1 en /users/2 et reçoit le profil de Bob.

FAIT OBSERVÉ

L'identité est valide, mais l'API n'a pas vérifié la propriété de l'objet.

RISQUE À TESTER

Cacher le bouton ou utiliser un UUID ne remplace pas le contrôle serveur.

Règle : une hypothèse utile indique l'observation qui pourrait la réfuter.

Vulnérable -> corrigé -> vérifié

ÉTAT VULNÉRABLE

- X Contrôle frontend
- X Rôle user seulement
- X Objet prévisible



ÉTAT CORRIGÉ

- OK Owner/admin côté API
- OK Deny par défaut
- OK Journal sans donnée sensible

Même test négatif + parcours légitime positif + risque résiduel explicite

Démo : Établir la matrice 200/401/403

TERMINAL LOCAL

```
$ curl -sS -o /dev/null -w '%{http_code}'  
' -H 'Authorization: Bearer TOKEN' http://127.0.0.1:8080/api/users/2
```

SORTIE ATTENDUE

200 en état vulnérable; 403 après correction; 401 sans identité; 200 pour l'objet propre

Prédire -> exécuter -> lire -> conclure -> retester -> reset

Checkpoint de compréhension

Pourquoi faut-il retester aussi l'objet d'Alice après la correction ?

1 · Position

Formulez votre réponse

2 · Preuve

Quelle observation la soutient ?

3 · Limite

Qu'est-ce qui reste non prouvé ?

Passage à la pratique supervisée

TP03: reproduire BOLA, modifier le contrôle, obtenir 200/401/403 et documenter le risque résiduel.

PREUVES À REMETTRE

1 baseline propriétaire

2 refus objet tiers

3 log authz expurgé

STOP CONDITION

**Cible inattendue
Secret réel
Reset incertain**

Validation et SQLi

Séparer strictement structure SQL et données, puis prouver la non-régression.

01

**Tracer source, validation
et sink**

02

**Expliquer la requête
paramétrée**

03

**Retester attaque et usage
légitime**

Palette pédagogique provisoire · charte officielle À confirmer

Trois repères avant l'outil

01

Le typage réduit les entrées possibles

Décision observable

02

Les paramètres séparent code et donnée

Limite à expliciter

03

Un filtre d'apostrophes est contournable

Preuve à conserver

Le flux et ses décisions



À chaque flèche : qui fait confiance à qui ? quelle donnée ? quel contrôle ? quelle preuve ?

Cas guidé : faits avant verdict

Une recherche concatène q dans SQL et un payload renvoie tout le catalogue.

FAIT OBSERVÉ

La variation du nombre de lignes établit un comportement injectable dans le lab.

RISQUE À TESTER

Supprimer certains caractères altère les données et laisse d'autres syntaxes.

Règle : une hypothèse utile indique l'observation qui pourrait la réfuter.

Vulnérable -> corrigé -> vérifié

ÉTAT VULNÉRABLE

- X Concaténation
- X Blacklist
- X Compte DB propriétaire



ÉTAT CORRIGÉ

- OK Paramètres driver
- OK Validation métier
- OK Rôle DB minimal

Même test négatif + parcours légitime positif + risque résiduel explicite

Démo : Comparer baseline, payload et requête paramétrée

TERMINAL LOCAL

```
$ curl -sS --get --data-urlencode "q=' OR '1'='1" http://127.0.0.1:8080/api/products
```

SORTIE ATTENDUE

plusieurs lignes avant; aucune ligne après; q=lamp reste fonctionnel

Prédire -> exécuter -> lire -> conclure -> retester -> reset

Checkpoint de compréhension

Pourquoi la validation seule ne remplace-t-elle pas les paramètres SQL ?

1 · Position

Formulez votre réponse

2 · Preuve

Quelle observation la soutient ?

3 · Limite

Qu'est-ce qui reste non prouvé ?

Passage à la pratique supervisée

TP04-A: SQLi locale, correction source, test négatif et recherche légitime.

PREUVES À REMETTRE

- 1 diff du sink
- 2 résultat avant/après
- 3 privilèges DB relevés

STOP CONDITION

**Cible inattendue
Secret réel
Reset incertain**

XSS, CSRF et CSP

Adapter le contrôle au contexte de sortie et au modèle d'envoi des credentials.

01

Distinguer validation et encodage

02

Expliquer CSRF et SameSite

03

Utiliser CSP en défense supplémentaire

Palette pédagogique provisoire · charte officielle À confirmer

Trois repères avant l'outil

01

**L'encodage dépend
du contexte
HTML/JS/URL**

Décision observable

02

**CSRF exploite
l'envoi automatique**

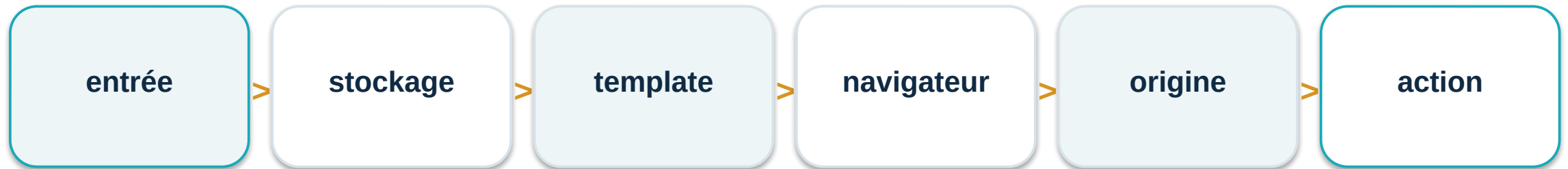
Limite à expliciter

03

**CSP ne répare pas
un sink dangereux**

Preuve à conserver

Le flux et ses décisions



À chaque flèche : qui fait confiance à qui ? quelle donnée ? quel contrôle ? quelle preuve ?

Cas guidé : faits avant verdict

Un commentaire est inséré dans `innerHTML`; le cookie est envoyé automatiquement.

FAIT OBSERVÉ

Le sink interprète du HTML contrôlé et l'action serveur peut manquer de preuve d'intention.

RISQUE À TESTER

Une CSP permissive ou SameSite seul laisse des chemins d'attaque.

Règle : une hypothèse utile indique l'observation qui pourrait la réfuter.

Vulnérable -> corrigé -> vérifié

ÉTAT VULNÉRABLE

- X innerHTML
- X CSRF sans token
- X CSP unsafe-inline



ÉTAT CORRIGÉ

- OK.textContent/encoding
- OK.token + origine
- OK.CSP restrictive mesurée

Même test négatif + parcours légitime positif + risque résiduel explicite

Démo : Comparer texte sûr et interprétation HTML

TERMINAL LOCAL

```
$ printf '%s\n' '<img src=x onerror=alert(1)>' | python3 -c 'import html,sys; print(html.escape(sys.stdin.read()))'
```

SORTIE ATTENDUE

les caractères actifs deviennent des entités; le contexte exact reste à vérifier

Prédire -> exécuter -> lire -> conclure -> retester -> reset

Checkpoint de compréhension

Pourquoi SameSite=Lax n'est-il pas une correction universelle de CSRF ?

1 · Position

Formulez votre réponse

2 · Preuve

Quelle observation la soutient ?

3 · Limite

Qu'est-ce qui reste non prouvé ?

Passage à la pratique supervisée

TP04-B + Quiz 2: cas XSS/CSRF locaux, correction, CSP et retest.

PREUVES À REMETTRE

- 1 sink identifié
- 2 requête CSRF comparée
- 3 CSP expliquée sans sur-promesse

STOP CONDITION

Cible inattendue
Secret réel
Reset incertain